

TRABAJO PRACTICO OBLIGATORIO

INTRODUCCIÓN A LA BIOINFORMÁTICA

Kin, Francisco - LU 1197791

Licenciatura en Bioinformática



FUNDACIÓN UADE
FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS

Índice

Índice general	2
1. Introducción	3
1.1. Bioinformática	3
1.2. Datos e información	3
1.2.1. Datos	3
1.2.2. Información	4
1.3. Estructura, secuencia y función de proteínas	4
1.3.1. Aminoácidos y proteínas	4
1.3.2. Proteínas globulares	5
1.3.3. Proteínas intrínsecamente desordenadas	6
1.3.4. Dominios proteicos	7
1.4. Adenovirus	7
1.5. Proteína E1A	8
1.6. Identidad de secuencias	8
1.7. Alineamiento Multiple de Secuencias (MSA)	9
1.8. Radio hidrodinámico	10
1.9. Kappa	10
1.10. AIUPred	11
1.11. Python	12
1.11.1. Biopython	12
1.11.2. LocalCIDER	13
1.12. UGENE	13
1.13. PyMol	13
1.14. RCSB PDB	14
2. Materiales y métodos	15
2.1. Unipro UGENE	15
2.2. PyMOL	15
2.3. Código Python	16
2.3.1. Dependencias	16
2.3.2. Alineamiento	17
2.3.3. Identidad	19
2.3.4. Contar aminoácidos	20
2.3.5. Cargas	21
2.3.6. Radio hidrodinámico	22
2.3.7. Kappa	23
2.3.8. AIUPred	24
3. Resultados	27
3.1. Datos primarios y secundarios	27
3.2. Radio Hidrodinámico	28
3.3. Divergencia en secuencias	29

4. Discusión de resultados	30
4.1. Análisis de datos	30
4.2. Longitud, Radio y Kappa	31
4.3. Regiones ordenadas y desordenadas	31
Referencias	32
Índice de figuras	33
Índice de Tablas	33

1. Introducción

1.1. Bioinformática

La bioinformática es una subdisciplina científica que utiliza tecnología informática para recolectar, almacenar, analizar y difundir datos biológicos, como secuencias de ADN y aminoácidos o anotaciones sobre estas secuencias. Los científicos y clínicos usan bases de datos que organizan e indexan esta información biológica para aumentar la comprensión sobre la salud y la enfermedad, y en algunos casos, como parte del cuidado médico.

El papel de la bioinformática en la investigación biológica es comparable al rol del análisis de datos en la era de la información y el Internet. Antes, el desafío principal era acceder a la información, pero los avances en la lectura de secuencias de ADN han reducido significativamente esa barrera. Ahora, el desafío es entender e interpretar la información recopilada. Dado que los conjuntos de datos son grandes, ya sea información sobre visitas a sitios web o el genoma humano, los métodos basados en computadoras son la opción predeterminada. En última instancia, el trabajo de la bioinformática con genomas humanos busca descubrir conocimientos prácticos sobre la salud y la biología humana en toda su complejidad (David Adams, 2024).

1.2. Datos e información

1.2.1. Datos

Los datos representan la forma más básica de información. Son símbolos, números, letras o cualquier otro tipo de representación que documenta un hecho, medición u observación. Por sí mismos, los datos carecen de contexto y significado. En bioinformática, los datos constituyen la materia prima que se utiliza para realizar análisis más profundos.

Datos primarios: Son datos originales y sin procesar que se obtienen directamente de experimentos o investigaciones. Ejemplos incluyen lecturas de secuenciación genómica, resultados de experimentos de expresión génica y medidas de actividad enzimática. En el contexto de proteínas, esto puede incluir secuencias de aminoácidos obtenidas mediante espectrometría de masas o estructuras tridimensionales obtenidas por cristalografía de rayos

X.

Datos secundarios: Son datos derivados de los datos primarios a través de análisis, interpretación y procesamiento adicionales. Ejemplos incluyen anotaciones de genomas, perfiles de expresión génica integrados y modelos estructurales de proteínas. En el caso de proteínas, esto puede incluir predicciones de estructuras secundarias y terciarias, así como anotaciones de funciones y interacciones proteicas.

1.2.2. Información

La información surge del procesamiento, organización e interpretación de los datos. A través de este proceso, los datos adquieren contexto y significado, lo que permite responder preguntas específicas, identificar patrones y tomar decisiones fundamentadas. En el ámbito de la bioinformática, transformar datos en información es crucial para avanzar en el conocimiento científico.

1.3. Estructura, secuencia y función de proteínas

1.3.1. Aminoácidos y proteínas

Las proteínas son polímeros biológicos cuyas subunidades son los aminoácidos (Davey *et al.*, 2012). La estructura de los aminoácidos consta de un carbono central α (C_α) al que está unido un grupo amino ($-NH_2$), un grupo carboxilo ($-COOH$) y un grupo R (Figura 1). El C_α de los aminoácidos es un centro asimétrico por lo tanto exhibe quiralidad (con excepción de la glicina), pudiendo existir como dos enantiómeros, L (levógiro) y D (dextrógiro), sin embargo, los aminoácidos proteinogénicos tienen configuración α -L-aminoácido.

El grupo R o cadena lateral define las características propias del aminoácido haciéndolo variar en estructura química, tamaño y carga eléctrica. En base a estas características se los puede clasificar por su naturaleza química dominante en polares o apolares (Figura 2) y por la presencia o ausencia de grupos ionizables. (Chemes *et al.*, 2015)

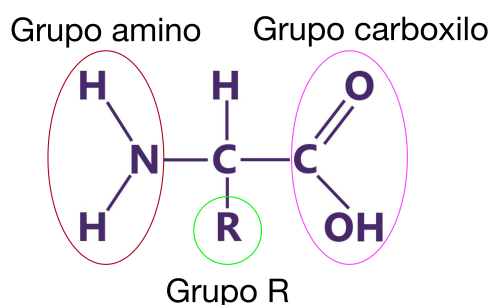


Fig. 1. Estructura de un aminoácido.

ÁMINOACIDOS POLARES				CADENA LATERAL	ÁMINOACIDO APOLARES				CADENA LATERAL
Ácido aspártico	Asp	D	negativa		Alanina	Ala	A	apolar	
Ácido glutámico	Glu	E	negativa		Glicina	Gly	G	apolar	
Arginina	Arg	R	positiva		Valina	Val	V	apolar	
Lisina	Lys	K	positiva		Leucina	Leu	L	apolar	
Histidina	His	H	positiva		Isoleucina	Ile	I	apolar	
Asparagina	Asn	N	polar no cargada		Prolina	Pro	P	apolar	
Glutamina	Gln	Q	polar no cargada		Fenilalanina	Phe	F	apolar	
Serina	Ser	S	polar no cargada		Metionina	Met	M	apolar	
Treonina	Thr	T	polar no cargada		Triptofano	Trp	W	apolar	
Tirosina	Tyr	Y	polar no cargada		Cisteína	Cys	C	apolar	

Fig. 2. Lista de aminoácidos diferenciados por polaridad y tipo de carga.

1.3.2. Proteínas globulares

Las proteínas globulares, o esferoproteínas son uno de los tres tipos principales de la clasificación de las proteínas por su forma (globulares, fibrosas y mixtas), diferenciándose fundamentalmente de las fibrosas por ser más o menos solubles en disoluciones acuosas (donde forman suspensiones coloidales), siendo las fibrosas prácticamente insolubles.

La naturaleza globular de estas proteínas se puede determinar sin los medios de las técnicas modernas, utilizando solamente ultracentrifugadoras o por técnicas de dispersión dinámica de la luz. La estructura terciaria de la proteína induce su esfericidad. Los grupos apolares (colas hidrofóbicas) se agrupan en el interior de la molécula, mientras que los polares (cabezas hidrofílicas) se disponen hacia el exterior, permitiendo la atracciones dipolo-dipolo con el disolvente, lo que explica la solubilidad de la molécula (Whitford, 2005).

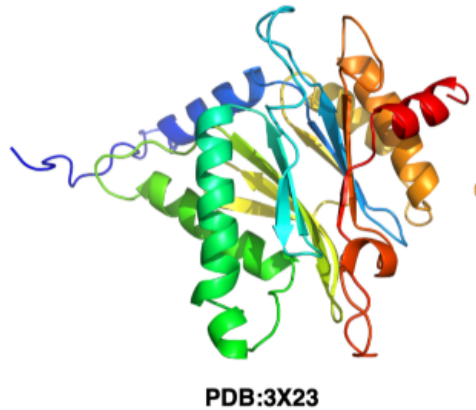


Fig. 3. Proteína Globular PDB:3X23

1.3.3. Proteínas intrínsecamente desordenadas

Las IDPs son proteínas o regiones de proteínas que carecen de una estructura tridimensional estable y bien definida en condiciones fisiológicas. En lugar de adoptar una conformación fija, las IDPs existen como conjuntos heterogéneos de conformaciones que cambian rápidamente. Este desorden estructural no es una anomalía, sino una característica intrínseca de estas proteínas, que les permite cumplir funciones biológicas específicas que requieren flexibilidad estructural(Uversky, 2014).

Muchas IDPs son capaces de unirse a múltiples blancos, y les permite adoptar diferentes estructuras y diversas funcionalidades, es decir, de una única secuencias se pueden dar múltiples estructuras, lo que brinda en muchos casos un espectro de acción más grande que el de una proteína globular(Glavina *et al.*, 2018).

Las IDPs y las regiones de proteínas intrínsecamente desordenadas (IDPRs) muestran una marcada heterogeneidad conformacional y constituyen una parte significativa del reino de las proteínas(Uversky, 2014).

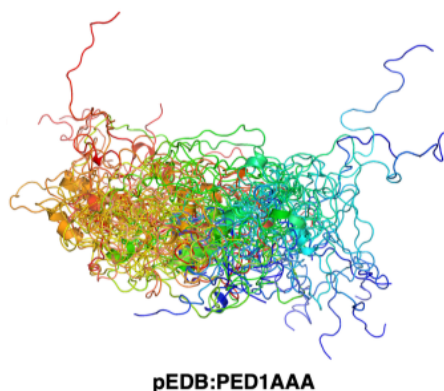


Fig. 4. Proteína intrínsecamente desordenada pEDB:PED1AAA

1.3.4. Dominios proteicos

El dominio proteico de una proteína puede ser definido de diversas formas: a) como cualquier región de la cadena que pueda plegarse en forma independiente del resto de la cadena polipeptídica; b) cualquier región que pueda evolucionar en forma independiente al resto de la cadena; c) la mínima unidad funcional que pueda soportar una función.

Sin embargo, en proteínas desordenadas un dominio proteico puede ser definido como una región conservada, funcional que puede evolucionar y existir de forma autónoma al resto de la cadena polipeptídica.

Un dominio proteico en proteínas desordenadas es capaz de adquirir una estructura tridimensional transitoria al unirse a blancos moleculares.

1.4. Adenovirus

Los adenovirus son una familia de virus que infectan a vertebrados, incluyendo a los humanos.

Las patologías producidas por los adenovirus son infecciones en las vías respiratorias, gastroenteritis, cistitis hemorrágica y conjuntivitis entre otras (Murray *et al.*, 2009). La familia Adenoviridae se divide en cinco géneros: aviadenovirus, atadenovirus, siadenovirus, ichtadenovirus y mastadenovirus, siendo este último exclusivo de mamíferos. Son virus sin envoltura, de forma deltaicosaédrica, con tamaño medio entre 70 y 90 nm (Fields *et al.*, 2013).

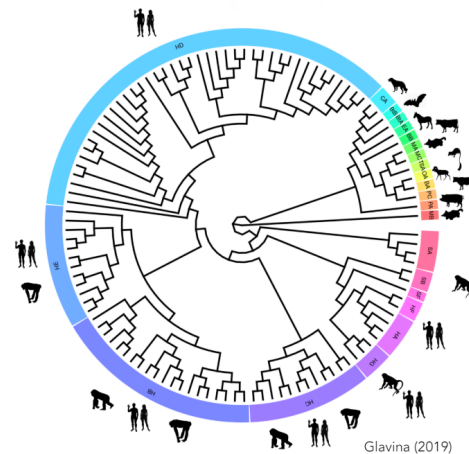


Fig. 5. árbol filogenético con los distintos géneros de adenovirus

1.5. Proteína E1A

La proteína desordenada E1A es esencial para la infección del virus, regula la transcripción de genes tanto del huésped como del virus y desregula el ciclo celular de la célula huésped induciendo condiciones que son adecuadas para la replicación viral. La proteína E1A es el determinante principal de la oncogenicidad del virus (Glavina *et al.*, 2018)

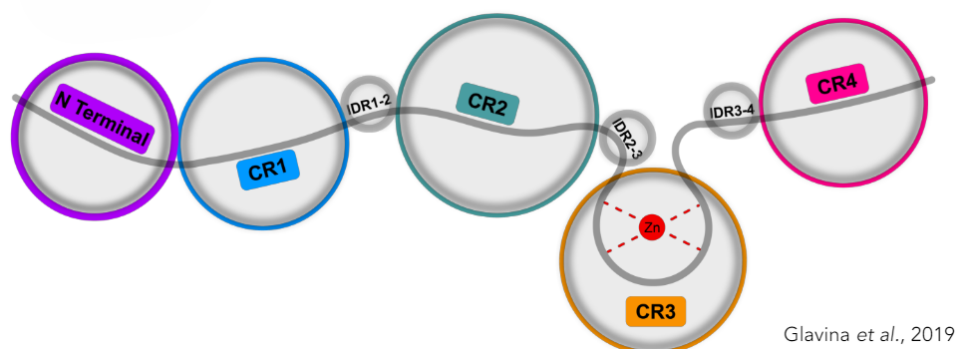


Fig. 6. Dominios proteicos de la proteína E1A

1.6. Identidad de secuencias

La identidad es una forma de cuantificar la similitud entre dos secuencias de aminoácidos, describiendo la fracción de posiciones que coinciden en un alineamiento de secuencias, donde las secuencias se escriben en filas, de modo tal que los residuos alineados aparezcan en columnas sucesivas. Las columnas alineadas que presentan el mismo aminoácido van a ser consideradas

como coincidentes, y las que no presenten el mismo aminoácido o presenten huecos van a ser consideradas como no coincidentes. La cantidad de coincidencias sobre la longitud del alineamiento va a indicarnos la identidad de secuencia.

Para calcular la identidad de secuencia entre una secuencia S1 y una secuencia S2 en un alineamiento de longitud L, se calcula como:

$$Id(S1, S2) = \frac{1}{K} \sum_{j=1}^L i_j \quad (1)$$

donde i toma valor 1 si ambos aminoácidos en la posición j son iguales y valor 0 si ambos aminoácidos son distintos o hay sitios vacíos en alguna de las dos secuencias, cualquiera de estas opciones es considerada como comparaciones positivas y las acumulamos en K a modo de contador. Las posiciones que son sitios vacíos en ambas secuencias no se incluyen en las comparaciones positivas.

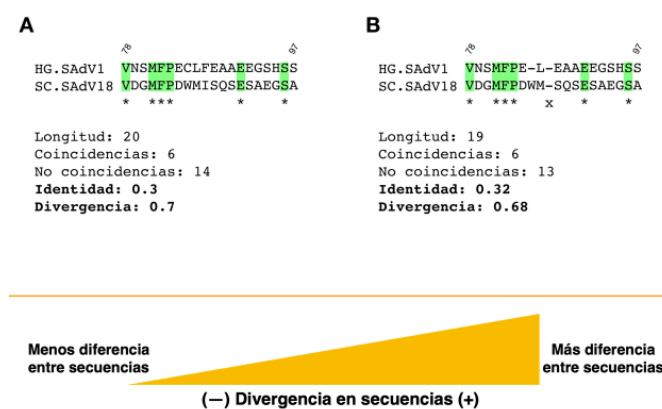


Fig. 7. Ejemplo de divergencia de secuencia para dos cadenas de aminoácidos

1.7. Alineamiento Múltiple de Secuencias (MSA)

Para poder comparar secuencias, ya sean peptídicas, ADN o ARN es necesario poder identificar las posiciones que son genéticamente equivalentes. En el caso de secuencias polipeptídicas, el alineamiento múltiple de secuencias (MSA) permite identificar posiciones ocupadas por aminoácidos iguales o con propiedades similares entre las distintas secuencias.

En muchos casos se asume que las secuencias a alinear tienen una relación evolutiva, es decir, que comparten un ancestro común.

En un alineamiento múltiple de secuencias cada secuencia de aminoácidos es escrita en una línea y se hacen coincidir aquellas posiciones genéticamente equivalentes insertando sitios vacíos donde sean necesarios. La inserción de sitios vacíos se hace para maximizar la similitud global entre pares de secuencias del alineamiento (Glavina *et al.*, 2018).

1.8. Radio hidrodinámico

El R_h es el radio de una esfera idealizada que se difundiría a la misma velocidad que la molécula de interés, en nuestro caso una proteína, y se basa en la relación de Stokes-Einstein.

$$R_h = \frac{k_b T}{6 \pi \eta D} \quad (2)$$

donde k_b es la constante de Boltzmann, T es la temperatura, η es la viscosidad y D es el coeficiente de difusión de traslación. En el caso de proteínas globulares o plegadas, R_h se encuentra en el orden de la dimensión de la proteína observada en una estructura cristalográfica, ya que las mismas tienen una forma compacta que aproxima bastante bien a una esfera.

Para las proteínas intrínsecamente desordenadas, se puede utilizar una forma mejorada de dicha ecuación, teniendo en cuenta la fracción de residuos de prolina (P_{pro}), la carga neta absoluta ($|Q|$) y la presencia de una etiqueta de polihistidina, donde A - D son constantes que se ajustan a partir de las pendientes (A y C) y los desplazamientos (B y D) de los ajustes lineales entre R_{rel} y P_{pro} o $|Q|$, y $Shis^*$ es un factor de escala aplicado si la proteína tiene una etiqueta de polihistidina ($Shis^*$ es 1 si no hay etiqueta presente).

$$R_h = (AP_{pro} + B)(C|Q| + D)Shis^*R_0N^v \quad (3)$$

Los parámetros de mejor ajuste para la Ecuación 3, basados en la bibliografía, son $A = 1.24$, $B = 0.904$, $C = 0.00759$, $D = 0.963$, $Shis^* = 0.901$, $R_0 = 2.49$ y $v = 0.509$. (Marsh y Forman-Kay, 2010)

1.9. Kappa

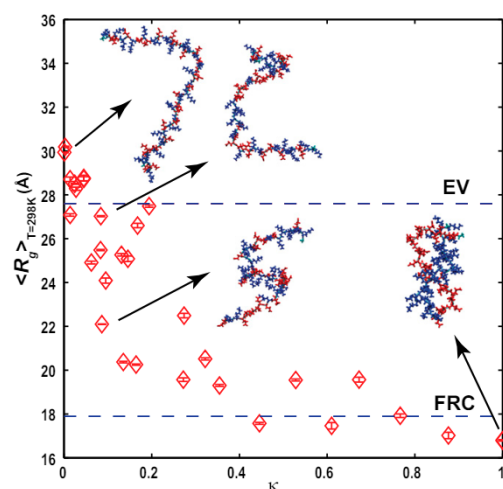
Kappa (k) (Das y Pappu, 2013) es un parámetro relacionado a la fracción de residuos cargados y su distribución en la estructura primaria de una proteína, que nos indica cuán

homogeneizada es la distribución de cargas de la estructura primaria. El parámetro toma valores entre cero y uno, siendo cero cuando las cargas se encuentran "mezcladas", y uno cuando la distribución de cargas están máximamente "segregada".

$$\kappa = \left(\frac{\delta}{\delta_{\max}} \right) \quad (4)$$

Label	Sequence	K
sv1	EE	0.0009
sv2	EE	0.0025
sv3	EE	0.0139
sv4	EE	0.0140
sv5	EE	0.0245
sv6	EE	0.0273
sv7	EE	0.0450
sv8	EE	0.0450
sv9	EE	0.0624
sv10	EE	0.0834
sv11	EE	0.0841
sv12	EE	0.0864
sv13	EE	0.0951
sv14	EE	0.1311
sv15	EE	0.1354
sv16	EE	0.1458
sv17	EE	0.1643
sv18	EE	0.1677
sv19	EE	0.1941
sv20	EE	0.2721
sv21	EE	0.2737
sv22	EE	0.3218
sv23	EE	0.3545
sv24	EE	0.4456
sv25	EE	0.5283
sv26	EE	0.6101
sv27	EE	0.6729
sv28	EE	0.7666
sv29	EE	0.8764
sv30	EE	1.0000

(a) Treinta variantes de secuencia para el sistema (Glu-Lys)25. La columna 1 muestra la etiqueta de cada variante de secuencia. La columna 2 muestra la secuencia real, con los residuos de Glu en rojo y los residuos de Lys en azul. La columna 3 muestra los valores de K.



(b) Promedios del radio de giro $\langle R_g \rangle$ para variantes de secuencia del sistema (Glu-Lys)25. Los recuadros muestran conformaciones representativas para cuatro variantes de secuencia. Las cadenas laterales de Glu se muestran en rojo, y las cadenas laterales de Lys se muestran en azul. Las dos líneas discontinuas intersectan la ordenada en valores de $\langle R_g \rangle$ esperados para todas las variantes de secuencia del sistema (Glu-Lys)25 modeladas en el límite EV o como enrollamientos aleatorios de Flory (FRCs).

Fig. 8. Comparación de variantes de secuencia y promedios del radio de giro para el sistema (Glu-Lys)25.

1.10. AIUPred

AIUPred es una herramienta innovadora para predecir el desorden intrínseco de las proteínas (IDP). Utiliza la estimación de energía y el aprendizaje profundo, mejorando el rendimiento sobre IUPred, su predecesor. Disponible como un servidor web, AIUPred permite a los usuarios introducir secuencias de proteínas y obtener predicciones y visualizaciones detalladas del desorden. Esta herramienta es valiosa para la investigación de IDP (Li *et al.*, 2024).

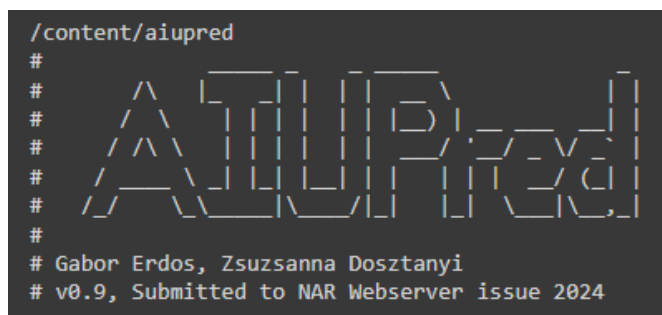


Fig. 9. Mensaje presentado en consola al usar el software.

1.11. Python

Para el desarrollo del trabajo practico, fue utilizado el lenguaje de programación Python. Python es un lenguaje de programación interpretado, orientado a objetos y de alto nivel con semántica dinámica. Sus estructuras de datos integradas, junto con la tipificación y vinculación dinámicas, lo hacen ideal para el desarrollo rápido de aplicaciones y scripting. La sintaxis simple y legible de Python reduce los costos de mantenimiento del código, y su soporte para módulos y paquetes fomenta la modularidad y reutilización del código. Python y su extensa biblioteca estándar están disponibles gratuitamente para todas las plataformas principales, lo que contribuye a su popularidad y alta productividad entre los programadores (Foundation, 2024).

1.11.1. Biopython

En el campo de la bioinformática, Python destaca especialmente gracias a bibliotecas como Biopython. Biopython es una biblioteca de bioinformática de código abierto en Python, inspirada en Bioperl y desarrollada desde agosto de 1999. Su objetivo es proporcionar herramientas eficientes para cálculos bioinformáticos, evitando esfuerzos repetidos en el desarrollo de software. Entre las diversas funcionalidades que ofrece Biopython, destaca la interfaz AlignIO, diseñada para trabajar con múltiples alineamientos de secuencias. AlignIO permite leer, escribir y manipular alineamientos en varios formatos de archivo comunes en bioinformática, como FASTA, Clustal, PHYLIP, Nexus y Stockholm, entre otros (Contributors, 2024).

1.11.2. LocalCIDER

LocalCIDER es un conjunto de recursos y herramientas computacionales diseñados para analizar y entender cómo las secuencias de aminoácidos de las proteínas intrínsecamente desordenadas influyen en sus propiedades conformacionales y funcionales. Esta herramienta permite a los investigadores realizar cálculos rápidos y accesibles sobre la composición de secuencias de IDPs y cómo estas secuencias pueden relacionarse con sus comportamientos biológicos Holehouse *et al.* (2017).

1.12. UGENE

UGENE es una plataforma integral de bioinformática diseñada para facilitar el análisis de datos biológicos. Esta herramienta de código abierto combina una interfaz gráfica de usuario intuitiva con una potente funcionalidad que permite a los investigadores realizar una amplia gama de tareas bioinformáticas. UGENE incluye módulos para el alineamiento de secuencias, análisis filogenético, predicción de genes, y visualización de estructuras moleculares, entre otros. Su capacidad para integrar diversas herramientas y formatos de datos, junto con su compatibilidad con sistemas operativos Windows, macOS y Linux, la convierte en una opción versátil y accesible para científicos de diferentes disciplinas. Con UGENE, es posible realizar análisis complejos de manera eficiente, contribuyendo así al avance de la investigación en genómica y biología molecular (Okonechnikov *et al.*, 2012).

1.13. PyMol

PyMOL es una herramienta de visualización molecular poderosa y ampliamente utilizada en bioquímica y biología estructural. Este software, conocido por su versatilidad y precisión, permite a los científicos visualizar y analizar estructuras tridimensionales de moléculas biológicas, como proteínas, ácidos nucleicos y complejos macromoleculares. PyMOL ofrece una variedad de funciones avanzadas, incluyendo la generación de gráficos de alta calidad, la manipulación interactiva de estructuras, y la capacidad de realizar análisis estructurales detallados. Además, su compatibilidad con numerosos formatos de archivos y su capacidad para integrarse con otras herramientas bioinformáticas lo hacen una elección preferida en

laboratorios de investigación de todo el mundo. PyMOL no solo facilita la comprensión de las relaciones estructurales y funcionales de las biomoléculas, sino que también proporciona un medio efectivo para comunicar hallazgos científicos a través de visualizaciones claras y precisas (Schrödinger, 2020).

1.14. RCSB PDB

El RCSB Protein Data Bank (PDB) proporciona acceso a una extensa base de datos de estructuras tridimensionales de macromoléculas biológicas, como proteínas y ácidos nucleicos. Ofrece herramientas para visualizar y analizar estas estructuras, facilitando la investigación en biología estructural y bioinformática. El sitio también incluye recursos educativos y documentación detallada sobre el depósito y uso de datos estructurales. El PDB es esencial para entender la función y la interacción de biomoléculas en el contexto de la salud y la enfermedad (PDB, 2024).

2. Materiales y métodos

2.1. Unipro UGENE

Utilizamos el progrmaa Unipro UGENE para poder visualizar nuestro archivo Fasta.

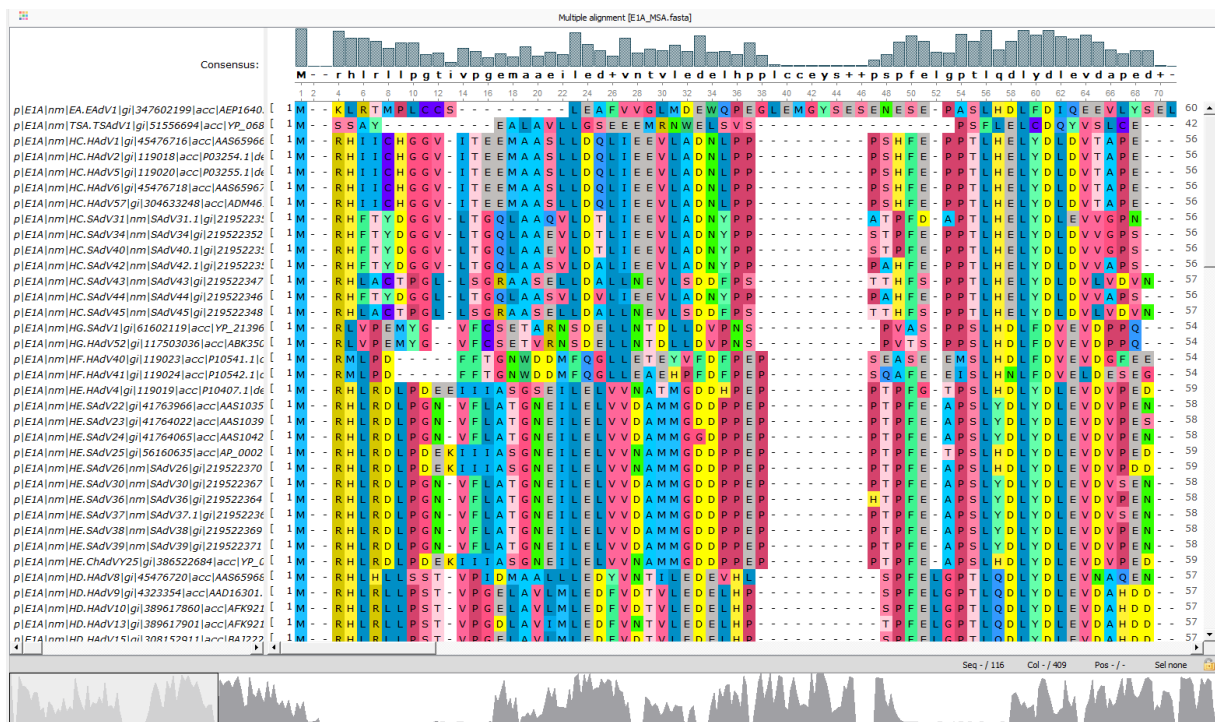


Fig. 10. Captura de pantalla mostrando una porción algunas de las cadenas incluidas en el archivo Fasta.

Este programa solo la usamos para la visualiacion de los datos, para el procesamiento de los datos utilizamos codigo python que se vera mas tarde.

2.2. PyMOL

También utilizamos el programa PyMOL para ver como visualizar estructuras 3D de proteínas. Como ejemplo usamos la proteína 6BB5

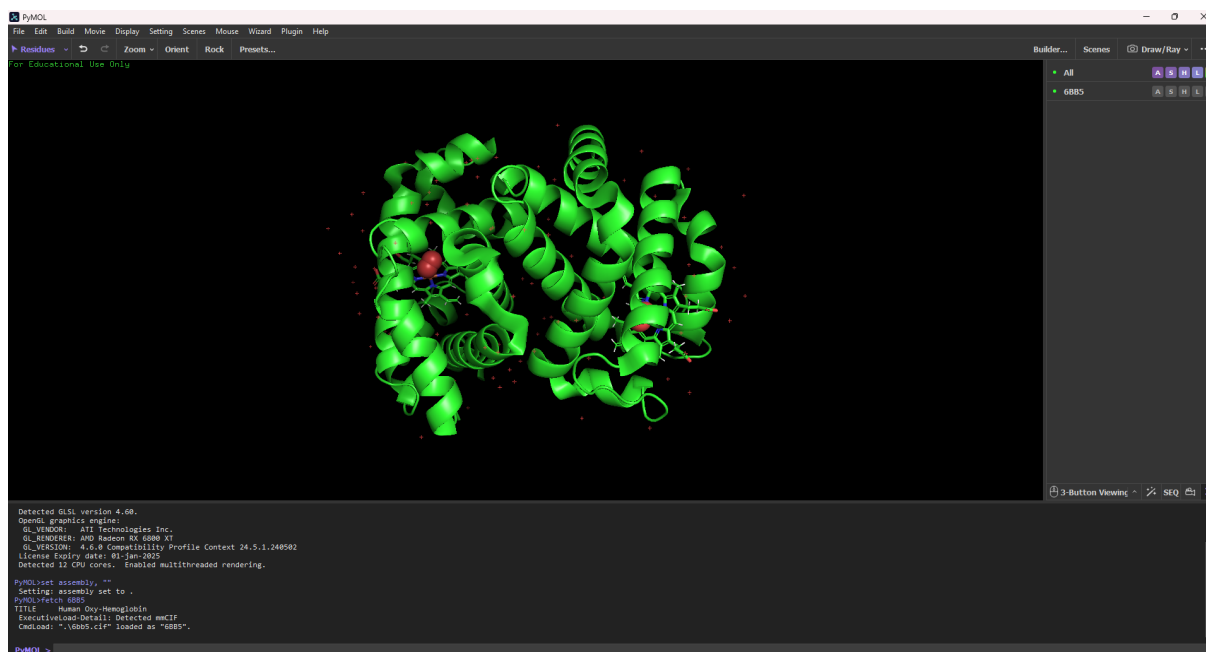


Fig. 11. Captura de pantalla mostrando la estructura tridimensional de la proteína 6BB5, utilizada como ejemplo.

Para hacerlo es posible utilizar el buscador incorporado en el programa, yendo a File→Get PDB y rellenar el campo de PDB ID con la proteína deseada

2.3. Código Python

2.3.1. Dependencias

El primer paso que realizamos, fue el de instalar mediante `pip` las distintas librerías externas que íbamos a utilizar

```
!pip install biopython
!pip install localcider
```

Luego de esto ya podemos importar las librerías y funciones. Durante el transcurso del TP fuimos aprendiendo paulatinamente como utilizarlas, pero para facilitar la presentación, se juntaron todas en el siguiente código:

```
from Bio import AlignIO
from localcider.sequenceParameters import SequenceParameters
```

```
from itertools import combinations
import matplotlib.pyplot as plt
```

De esta manera importamos AlignIO de Biopython para hacer el alineamiento, SequenceParameters de localcider.sequenceParameters para poder utilizar Kappa, combinations de itertools para ver una forma alternativa de generar combinaciones de cadenas y finalmente, plt de matplotlib.pyplot para poder graficar los datos de radio y kappa vs largo de secuencias.

2.3.2. Alineamiento

Para alinear las distintas cadenas de proteínas, utilizamos la biblioteca Biopython, empleando la función AlignIO de la siguiente manera:

```
alineamiento = AlignIO.read(open(E1A_FILE), "fasta")
```

Especificando el archivo a usar y el formato, en este caso, fasta.

Luego utilizamos la siguiente linea de código para poder calcular el promedio de

```
suma = 0
contador = 0
```

```
for registro in alineamiento:
    print(registro.id.split(sep="|")[3] + "\t" + str(len(registro.seq.replace("-", ""))))
    suma += len(registro.seq.replace("-", ""))
    contador += 1
```

```
promedio = suma / contador
```

```
print (promedio)
```

este código recorre una la lista alineamiento, imprime un identificador y la longitud de la secuencia sin guiones de cada registro, y finalmente calcula e imprime el promedio de las longitudes de estas secuencias.

Otra forma de conocer la longitud es usando `len` y reemplazando los gaps por espacios vacíos:

```
len(alineamiento[0].seq.replace("-", " "))
```

Para guardar estos datos en un archivo cvs, utilice el siguiente código:

```
def guardarLongitudes(alineamiento):  
  
    csv = open("resultados_Longitudes.csv", "w")  
  
    suma = 0  
    contador = 0  
  
    for registro in alineamiento:  
        csv.write(registro.id.split(sep="|")[3] + ";" + str(len(registro.  
        suma += len(registro.seq.replace("-", " "))  
        contador += 1  
  
    promedio = suma / contador  
  
    csv.write("Promedio;;" + str(promedio))  
  
    csv.close()
```

A medida que se recorre el alineamiento, la función `write` va escribiendo las longitudes de cada cadena. finalmente termino escribiendo el promedio y cerrando el archivo csv.

2.3.3. Identidad

Para calcular la identidad entre dos cadenas usamos:

```
def identidad (seq1, seq2):  
    matcheo = 0  
    mismacheo = 0  
  
    longitud = len(seq1)  
  
    for pos in range(longitud):  
        if(seq1[pos] != "-" and seq2[pos] != "-"):  
            if (seq1[pos] == seq2[pos]):  
                matcheo += 1  
            else:  
                mismacheo += 1  
  
    return (matcheo / (matcheo + mismacheo))
```

Esta función compara dos secuencias y devuelve el grado de divergencia entre ellas. Para lograr esto, utiliza un ciclo `for` que verifica posición por posición si existe coincidencia entre las cadenas en la posición correspondiente.

Para imprimir las identidades entre todas los alineamientos, primero programamos una implementación propia y luego probamos la función `combinations` de la biblioteca `itertools`. ambas formas dan el mismo resultado.

```
for i in range(len(alineamiento)-1):  
    for j in range(i+1, len(alineamiento)):  
        print(alineamiento[i].id.split(sep="|")[3] + "\t" \  
              + alineamiento[j].id.split(sep="|")[3] + "\t" \  
              + str(identidad(alineamiento[i].seq, alineamiento[j].seq)))  
  
combinado = list(combinations(alineamiento, 2))  
for reg in combinado:
```

```
print(reg[0].id.split(sep="|")[3] + "\t" + \
      reg[1].id.split(sep="|")[3] + "\t" + \
      str(identidad(reg[0].seq, reg[1].seq)))
```

Cabe aclarar, que la primera función es más rápida principalmente porque genera las combinaciones sobre la marcha y no requiere almacenamiento adicional, lo que la hace más eficiente en términos de uso de memoria y tiempo de ejecución, especialmente para listas grandes.

2.3.4. Contar aminoácidos

Para contar los aminoácidos usamos:

```
def contar_aminoacidos(seq):
    conteo = {}

    for aa in seq:
        if aa in conteo:
            conteo[aa] += 1
        else:
            conteo[aa] = 1

    return conteo
```

Esta función devuelve un diccionario con el conteo de cada aminoácido en una secuencia, incluidos los gaps. Para lograr esto, utiliza un ciclo `for` que recorre la secuencia: si un aminoácido aparece por primera vez, se añade al diccionario con un conteo inicial, y si ya está presente, incrementa su conteo en uno.

Para sacar la cantidad total de aminoácidos de todas las cadenas, se puede utilizar el siguiente código.

```
def contar_aminoacidos_totales(alineamiento):
    conteo = {}
```

```
for i in range(len(alineamiento)):
    for aa in alineamiento[i].seq:
        if aa in conteo:
            conteo[aa] += 1
        else:
            conteo[aa] = 1
    return conteo
```

con un doble ciclo `for`, se cuentan cada uno de los aminoácidos por secuencia del alineamiento. y para guardar los datos en un archivo csv.

```
documento = open("contar_aminoacidos_promedio.csv", "w")
texto = str(contar_aminoacidos_totales(alineamiento))
documento.write(texto)
documento.close()
```

2.3.5. Cargas

Para contar la cantidad de aminoácidos con carga usamos:

```
def calcular_cargas(sequencia):
    cargas = {'+': 0, '-': 0, 'neutro': 0}
    aminoacidos_cargados = {'R': '+', 'K': '+', 'D': '-', 'E': '-', \
        'H': '+', 'C': 'neutro', 'Y': 'neutro'}
    for aminoacido in sequencia:
        if aminoacido in aminoacidos_cargados:
            carga = aminoacidos_cargados[aminoacido]
            cargas[carga] += 1
    return cargas
```

Esta función calcula la cantidad de aminoácidos con cargas positivas, negativas y neutras en una secuencia de aminoácidos. Utiliza un diccionario `aminoacidos_cargados` para asignar una carga específica a ciertos aminoácidos ('+', '-', 'neutro'). Luego, recorre la secuencia

con un ciclo `for` y, si un aminoácido está presente en el diccionario, incrementa el conteo correspondiente en el diccionario cargas. Finalmente, devuelve el diccionario con el conteo de cada tipo de carga.

Y para calcular la carga total neta:

```
def carga_neta(sequencia):
    carga = 0
    aminoacidos_cargados = {'R': 1, 'K': 1, 'D': -1, 'E': -1}
    for aminoacido in sequencia:
        if aminoacido in aminoacidos_cargados:
            carga += aminoacidos_cargados[aminoacido]
    return carga
```

Esta función calcula la carga neta de una secuencia de aminoácidos. Utiliza un diccionario que asigna cargas a ciertos aminoácidos y un ciclo `for` para recorrer la secuencia. Si un aminoácido está en el diccionario, se suma su carga a una variable acumulativa.

2.3.6. Radio hidrodinámico

Para calcular el radio hidrodinámico usamos:

```
def radio(sequencia):
    prol = contar_aminoacidos(sequencia) ["P"]
    carga = carga_neta(sequencia)
    longitud = len(sequencia.replace("-", ""))

    rh = (1.24 * prol / longitud + 0.904) * (0.00759 * abs(carga) \
    + 0.963) * 2.49 * 0.901 * longitud ** 0.509

    return rh
```

Esta función primero cuenta la cantidad de prolina en la secuencia, calcula la carga neta y la longitud de la secuencia sin los caracteres "-". Utiliza estos valores en una fórmula que combina la proporción de prolina, la carga neta y la longitud para obtener el radio hidrodinámico.

2.3.7. Kappa

Para calcular el kappa usamos:

```
def kappa (secuencia):  
    seq = SequenceParameters(str(secuencia).replace("-", ""))  
    return seq.get_kappa()
```

Esta función elimina los caracteres "-" de la secuencia y crea un objeto `SequenceParameters` con la secuencia resultante. Luego, llama al método `get_kappa()` de este objeto para obtener el valor de kappa. esto es posible gracias a la biblioteca `localcider`. Finalmente, si deseo obtener los datos previamente calculados en un archivo `cvs`, utilizo la siguiente funcion:

```
def resultadosPrimarios (alineamiento):  
    documento = open("resultados_primarios.csv", "w")  
  
    for seq in alineamiento:  
        nombre = seq.id.split(sep="|")[3]  
        l = str(len(seq.seq.replace("-", "")))  
        r = str(radius(seq.seq))  
        k = str(kappa(seq.seq))  
  
        texto = nombre + ";" + l + ";" + r + ";" + k + "\n"  
  
        documento.write(texto)  
  
    documento.close()
```

Utilizando la funcion `write` como se explico previamente puedo ir guardando cada uno de los dato del alineamiento en mi documento.

2.3.8. AIUPred

Hay que tener en cuenta, que el Aiupred posee un limite de datos que puede analizar, por lo que para completar el procesamiento tuvimos que crear dos archivos por análisis. Una vez esto hecho esto, se crearon 4 variables para los 4 archivos generados

```
E1A_MSA1 = "/content/aiupred/E1A_MSA1.fasta"  
E1A_MSA2 = "/content/aiupred/E1A_MSA2.fasta"  
E1A_MSA_sin_gaps1 = "/content/aiupred/E1A_MSA_sin_gaps1.fasta"  
E1A_MSA_sin_gaps2 = "/content/aiupred/E1A_MSA_sin_gaps2.fasta"
```

Luego, con la siguiente linea de comandos, descomprimos y utilizamos el programa AIUPred para poder generar nuestros datos de tendencia de desorden.

```
!unzip /content/drive/MyDrive/aiupred.zip -d ./aiupred  
%cd /content/aiupred  
!python aiupred.py -i $E1A_MSA_sin_gaps1 -o /content/resultadosE1A_sin_gaps1.txt  
!python aiupred.py -i $E1A_MSA_sin_gaps2 -o /content/resultadosE1A_sin_gaps2.txt  
!python aiupred.py -i $E1A_MSA1 -o /content/resultadosE1A_con_gaps1.txt  
!python aiupred.py -i $E1A_MSA2 -o /content/resultadosE1A_con_gaps2.txt  
%cd /content
```

Estos archivos se combinaron nuevamente en resultadosE1A_sin_gaps.txt y resultadosE1A_con_gaps.txt.

```
E1A_FILE = "D:/Mis documentos/UADE/3-Introducción a la bioinformatica/E1A  
E1A_AIUPRED = "D:/Mis documentos/UADE/3-Introducción a la bioinformatica/  
ALINEAMIENTO_DESORDEN = "ALINEAMIENTO_DESORDEN.csv"  
ALINEAMIENTO_DESORDEN_GAPS = "ALINEAMIENTO_DESORDEN_GAPS.csv"
```

```

with open(ALINEAMIENTO_DESORDEN, 'w') as grabar:
    with open(E1A_AIUPRED, 'r') as archivo:
        for linea in archivo:
            if linea:
                if linea[0] != '\n' and linea[0] != '#':
                    if linea[0] == '>':
                        grabar.write('\n' + linea.split(sep="|")[3])
                    else:
                        grabar.write('; ' + linea.split()[2])

```

La función lee el archivo de entrada (E1A_AIUPRED), procesa sus líneas para extraer los datos, y los guarda en un nuevo archivo ((ALINEAMIENTO_DESORDEN)). Extrae los nombres de secuencias a partir de líneas de encabezado y los valores numéricos de predicciones de desorden, formateándolos con separadores específicos. Una vez que tenemos este archivo, procedemos con el siguiente código.

```

with open(ALINEAMIENTO_DESORDEN, 'r', newline='') as archivo:
    lector = csv.reader(archivo, delimiter=';')
    next(lector) #salto la primera linea que tengo en blanco
    lineas = list(lector)
    archivo.close()

```

```

with open(ALINEAMIENTO_DESORDEN_GAPS, 'w') as grabar:
    j = 0
    for secu in alineamiento:
        grabar.write('\n' + secu.id.split(sep='|')[3])
        caminador = 1
        for pos in secu:
            if pos != '-':
                grabar.write('; ' + lineas[j][caminador])
                caminador += 1
            else:

```

```
grabar.write(';')  
  
j += 1
```

La primera parte la utilizo para leer el archivo previamente creado ALINEAMIENTO_DESORDEN, una vez hecho esto, puedo pasar a crear un nuevo archivo ALINEAMIENTO_DESORDEN_GAPS, en el cual se agregan los gaps utilizando la funcion `write(';')`.

3. Resultados

3.1. Datos primarios y secundarios

Aminoácido	Cantidad	Promedio
-	17997	155.15
E	3374	29.087
P	3173	27.35
L	3000	25.86
D	2658	22.91
S	2347	20.23
V	1808	15.59
A	1696	14.62
G	1556	13.41
R	1373	11.84
T	1240	10.69

(a) parte 1

Aminoácido	Cantidad	Promedio
C	963	8.30
I	942	8.12
F	849	7.32
N	798	6.88
H	798	6.88
Q	788	6.80
K	701	6.04
M	686	5.91
Y	639	5.50
W	58	0.5
Aminoácidos Totales	29447	253.85

(b) parte 2

Tabla I. Resultados de `contar_aminoacidos_totales(alineamiento)`

Primera Secuencia	Segunda Secuencia	Divirgencia
HG.HAdV52	HB.SAdV46	0.4330357142857143
HG.HAdV52	HB.SAdV47	0.4375
HG.HAdV52	HD.HAdV28	0.4052863436123348
HG.HAdV52	HD.HAdV29	0.4088888888888889
HG.HAdV52	HD.HAdV30	0.43612334801762115
HG.HAdV52	HD.HAdV32	0.4044444444444444
HG.HAdV52	HD.HAdV33	0.43612334801762115
HG.HAdV52	HD.HAdV36	0.40969162995594716
HG.HAdV52	HD.HAdV37	0.41409691629955947
HG.HAdV52	HD.HAdV38	0.4008810572687225

Tabla II. 10 resultados de muestra sobre la divergencia entre dos secuencias utilizando nuestro código, como tambien implementando la función `combinations`

3.2. Radio Hidrodinámico

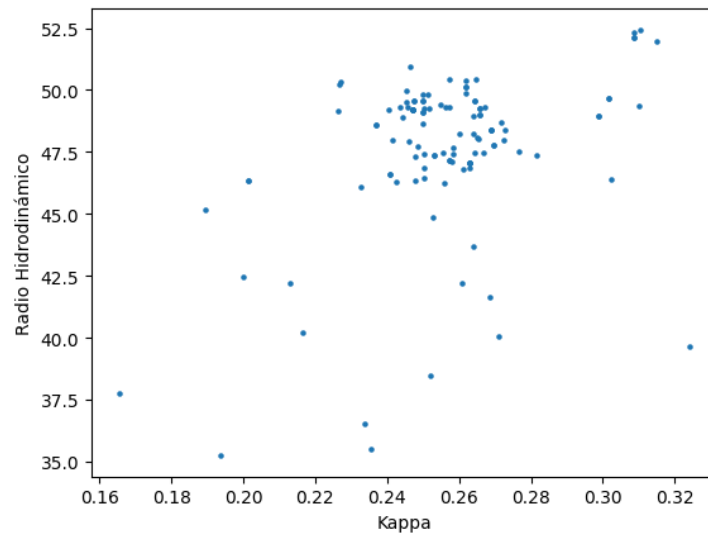


Fig. 12. Gráfico que muestra la relación obtenida entre el Kappa y el radio hidrodinámico.

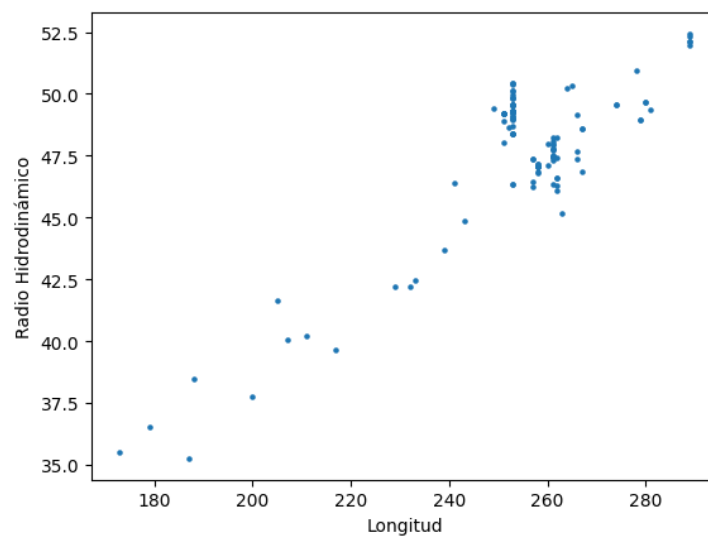


Fig. 13. Gráfico que muestra la relación obtenida entre la longitud de secuencia y el radio hidrodinámico.

3.3. Divergencia en secuencias

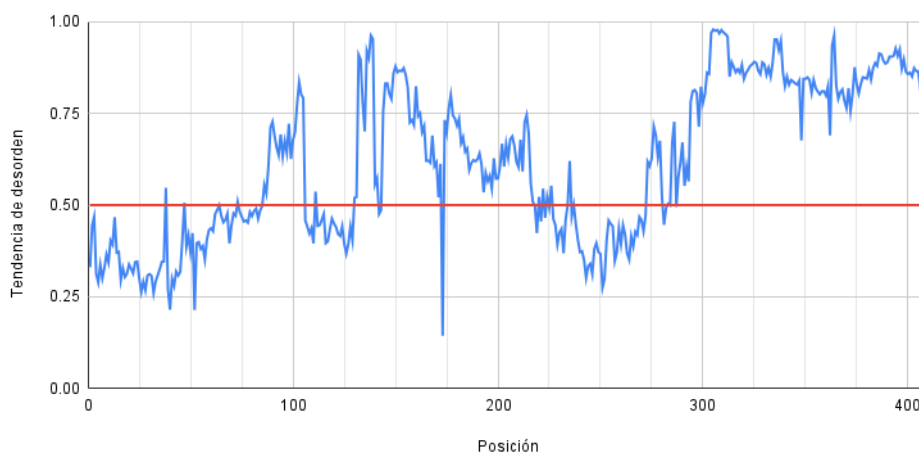


Fig. 14. Gráfico que muestra la relación obtenida entre la tendencia de desorden y la posición de aminoácido.

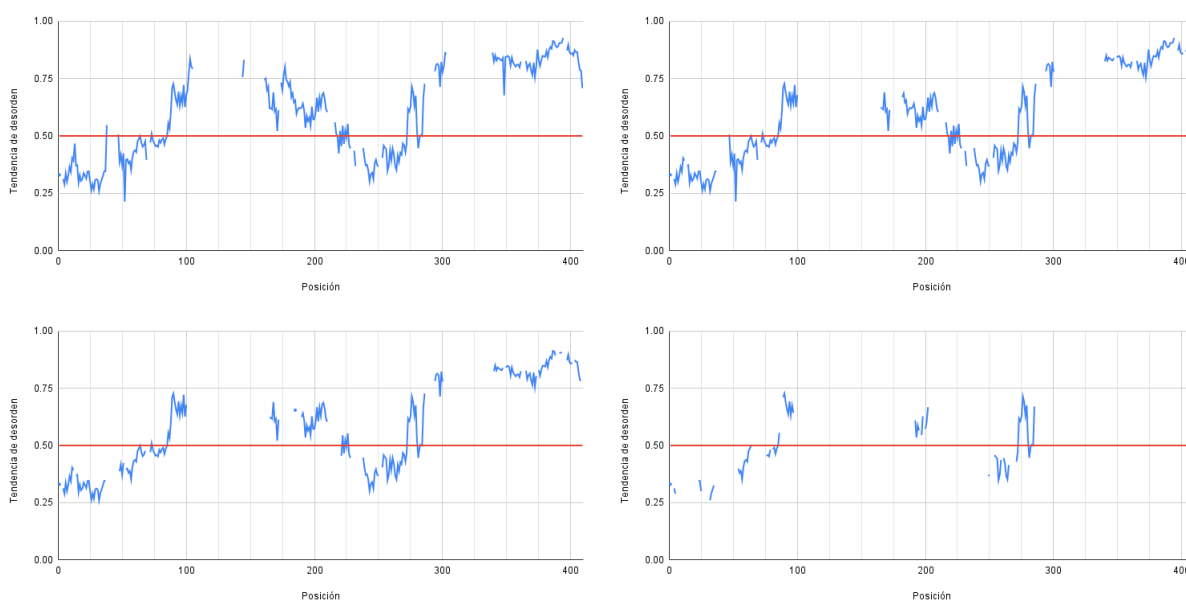


Fig. 15. Comparación de gráficos variando el threshold de aminoácidos en común, de arriba a la izquierda en sentido de agujas del reloj, por posición en 15, 50, 90 y 115.

4. Discusión de resultados

Gracias a la experiencia adquirida de las diversas herramientas y técnicas en bioinformática y análisis de datos biológicos, hemos podido explorar y analizar una variedad de datos primarios y secundarios en nuestro estudio. Utilizamos LaTeX para asegurar la presentación precisa y profesional de nuestros resultados científicos. Además, aprendimos como dar uso de Google Colab, integrando eficientemente bibliotecas de Python especializadas en el análisis de datos biológicos. Gracias a esto fue posible obtener una amplia cantidad de información sobre la proteína E1A y su alineamiento de secuencias múltiples.

4.1. Análisis de datos

Como se puede ver en la tabla I, fue posible obtener la cantidad de aminoácidos por secuencia. Estos datos proporcionan una visión general de la composición de las secuencias estudiadas. La abundancia de ciertos aminoácidos y la presencia de numerosos gaps pueden tener implicaciones importantes para la estructura, función, y estabilidad de la proteína. Este análisis puede ser útil para entender mejor las propiedades bioquímicas y biológicas de la proteína en cuestión.

Por ejemplo, una alta cantidad de gaps indica regiones de la secuencia no alineadas, sugiriendo variabilidad o inserciones/deleciones significativas. También posee en promedio una elevada cantidad de Ácido Glutámico, el cual posee carga negativa, crucial en catálisis enzimática y unión de iones metálicos. Importante para interacciones con proteínas huésped y actividad viral. La baja frecuencia de triptófano (W), un aminoácido voluminoso y aromático, podría indicar que no es necesario o es desfavorecido en la estructura de esta proteína particular.

En la tabla II se puede ver una porción de las combinaciones obtenidas, el cual posee un total de 6780 combinaciones. Este análisis de divergencia es útil porque nos permite identificar las diferencias y similitudes entre las secuencias de proteínas del adenovirus. Comprender la divergencia entre secuencias es esencial para estudiar la evolución de las proteínas, identificar regiones conservadas que pueden ser importantes para la función proteica y detectar posibles mutaciones que podrían afectar la virulencia o la respuesta inmune.

4.2. Longitud, Radio y Kappa

Viendo el gráfico de 12, se puede apreciar un cumulo de proteínas con kappa entre 0.24 y 0.28, este grupo tiene un tamaño promedio de 49 amstrongs de radio.

En el gráfico 13, también se puede observar como la mayoría de cadenas se agrupan en la longitud 260 aproximadamente

Un patrón peculiar que se puede observar al comparar ambos gráficos es que, en varios casos, las proteínas de mayor longitud, pero con un radio hidrodinámico similar, presentan un valor de Kappa menor en comparación con proteínas de menor longitud.

Una posible explicación para este fenómeno es que las proteínas más largas tienden a adoptar configuraciones más extendidas o desordenadas, lo que puede reducir la compacidad y aumentar la flexibilidad de la cadena polipeptídica. Esto puede resultar en un valor de Kappa menor. Por otro lado, las proteínas más cortas, con un radio hidrodinámico similar, podrían estar más restringidas en su conformación y, por lo tanto, ser más propensas a formar estas estructuras secundarias ordenadas, reflejándose en un valor de Kappa mayor.

4.3. Regiones ordenadas y desordenadas

como se puede apreciar en la figura 15, al tomar distintas cantidades mínimas de aminoácidos en común, se comienzan a visualizar los distintos dominios proteicos compartidos entre todas las secuencias analizadas. Recordando que si la tendencia al desorden en una posición es menor a 0.5, se puede considerar esta zona como ordenada.

La primera zona que se puede visualizar es de 0 a 75 aproximadamente. Se puede deducir que esta es la región N-terminal de la proteína. Esta región es conocida por su estabilidad, probablemente debido a la formación de estructuras secundarias bien definidas como hélices alfa o láminas beta, que confieren rigidez y resistencia a la degradación.

Otra zona ordenada se encuentra entre las posiciones 225 y 275, la cual probablemente corresponde al dominio CR3, que contiene un enlace de zinc. Este enlace de zinc proporciona estabilidad al dominio, permitiéndole mantenerse constante a lo largo del tiempo y conservado en las distintas isoformas de la proteína E1A.

Referencias

- Chemes, L. B., de Prat-Gay, G., y Sánchez, I. E. (2015). *Convergent evolution and mimicry of protein linear motifs in host-pathogen interactions*. doi:10.1016/j.sbi.2015.03.004.
- Contributors, B. P. (2024). *Biopython*. Accessed on June 30, 2024.
- Das, R. K. y Pappu, R. V. (2013). *Conformations of intrinsically disordered proteins are influenced by linear sequence distributions of oppositely charged residues*. Proceedings of the National Academy of Sciences. doi:10.1073/pnas.1304749110.
- Davey, N. E., Van Roey, K., Weatheritt, R. J., Toedt, G., Uyar, B., Altenberg, B., Budd, A., Diella, F., Dinkel, H., y Gibson, T. J. (2012). *Attributes of short linear motifs*. Molecular bioSystems. doi:10.1039/c1mb05231d.
- David Adams, M.D., P. (2024). *Bioinformática*. Página web.
- Fields, B. N., Knipe, D. M. D. M., y Howley, P. M. (2013). *Fields virology*. ISBN 1451105630.
- Foundation, P. S. (2024). *Python*. Accessed on June 30, 2024.
- Glavina, J., Román, E. A., Espada, R., de Prat-Gay, G., Chemes, L. B., y Sánchez, I. E. (2018). *Interplay between sequence, structure and linear motifs in the adenovirus E1A hub protein*. Virology, 525. doi:10.1016/j.virol.2018.08.012.
- Holehouse, A. S., Das, R. K., Ahad, J. N., Richardson, M. O. G., y Pappu, R. V. (2017). *CIDER: Resources to analyze sequence-ensemble relationships of intrinsically disordered proteins*. Biophysical Journal, 112:16–21.
- Li, M., Wang, J., Tang, H., y Zhang, Q. (2024). *AIUPred: prediction of intrinsically disordered proteins using energy estimation and deep learning*. Nucleic Acids Research. doi:https://doi.org/10.1093/nar/gkae385.
- Marsh, J. A. y Forman-Kay, J. D. (2010). *Sequence determinants of compaction in intrinsically disordered proteins*. Biophysical journal. doi:10.1016/j.bpj.2010.02.006.
- Murray, P. R., Rosenthal, K. S., y Pfaller, M. A. (2009). *Microbiología médica*. Elsevier España, 6ta edition. ISBN 978-84-8086-465-7.
- Okonechnikov, K., Golosova, O., Fursov, M., Varlamov, A., Vaskin, Y., Efremov, I., Grehov, O. G. G., Kandrov, D., Rasputin, K., Syabro, M., y Tleukenov, T. (2012). *Unipro UGENE: A unified bioinformatics toolkit*. doi:10.1093/bioinformatics/bts091.
- PDB, R. (2024). *RCSB Protein Data Bank*. Accessed on June 30, 2024.
- Schrödinger, L. (2020). *The PyMOL Molecular Graphics System, Version 3.0*.
- Uversky, V. N. (2014). *Intrinsically Disordered Proteins*.
- Whitford, D. (2005). *Proteins: Structure and Function*. Wiley, 1 edition. ISBN 0471498947; 9780471498940.

Índice de figuras

1.	Estructura de un aminoácido.	5
2.	Lista de aminoácidos diferenciados por polaridad y tipo de carga.	5
3.	Proteína Globular PDB:3X23	6
4.	Proteína intrínsecamente desordenada pEDB:PED1AAA	7
5.	árbol filogenético con los distintos géneros de adenovirus	8
6.	Dominios proteicos de la proteína E1A	8
7.	Ejemplo de divergencia de secuencia para dos cadenas de aminoácidos	9
8.	Comparación de variantes de secuencia y promedios del radio de giro para el sistema (Glu-Lys)25.	11
9.	Mensaje presentado en consola al usar el software.	12
10.	Captura de pantalla mostrando una porción algunas de las cadenas incluidas en el archivo Fasta.	15
11.	Captura de pantalla mostrando la estructura tridimensional de la proteína 6BB5, utilizada como ejemplo.	16
12.	Gráfico que muestra la relación obtenida entre el Kappa y el radio hidrodinámico.	28
13.	Gráfico que muestra la relación obtenida entre la longitud de secuencia y el radio hidrodinámico.	28
14.	Gráfico que muestra la relación obtenida entre la tendencia de desorden y la posición de aminoácido.	29
15.	Comparación de gráficos variando el threshold de aminoácidos en común, de arriba a la izquierda en sentido de agujas del reloj, por posición en 15, 50, 90 y 115.	29

Índice de tablas

I.	Resultados de <code>contar_aminoacidos_totales</code> (alineamiento) . . .	27
II.	10 resultados de muestra sobre la divergencia entre dos secuencias utilizando nuestro código, como tambien implementando la función <code>combinations</code> . .	27